

Manual Tecnico - Compilador GoLite

Organizacion de Lenguajes y Compiladores 1

Escuela de Vacaciones Junio 2026

Tabla de Contenidos

- 1. [Descripcion General](#)
- 2. [Arquitectura del Sistema](#)
- 3. [Analisis Lexico \(JFlex\)](#)
- 4. [Analisis Sintactico \(CUP\)](#)
- 5. [Arbol de Sintaxis Abstracta \(AST\)](#)
- 6. [Analisis Semantico \(VisitorSemantico\)](#)
- 7. [Ejecucion \(VisitorEjecucion\)](#)
- 8. [Sistema de Valores en Tiempo de Ejecucion](#)
- 9. [Manejo de Errores](#)
- 10. [Interfaz Grafica \(GUI\)](#)
- 11. [Reportes](#)
- 12. [Gramatica en BNF](#)
- 13. [Compilacion y Ejecucion](#)
- 14. [Estructura del Proyecto](#)

1. Descripcion General

GoLite es un subconjunto del lenguaje de programacion Go, disenado para fines academicos en el curso de Organizacion de Lenguajes y Compiladores 1. El compilador esta implementado en **Java 21** utilizando **JFlex** como generador de analizadores lexicos y **CUP 11b (vbmacher)** como generador de analizadores sintacticos.

El compilador realiza las siguientes fases:

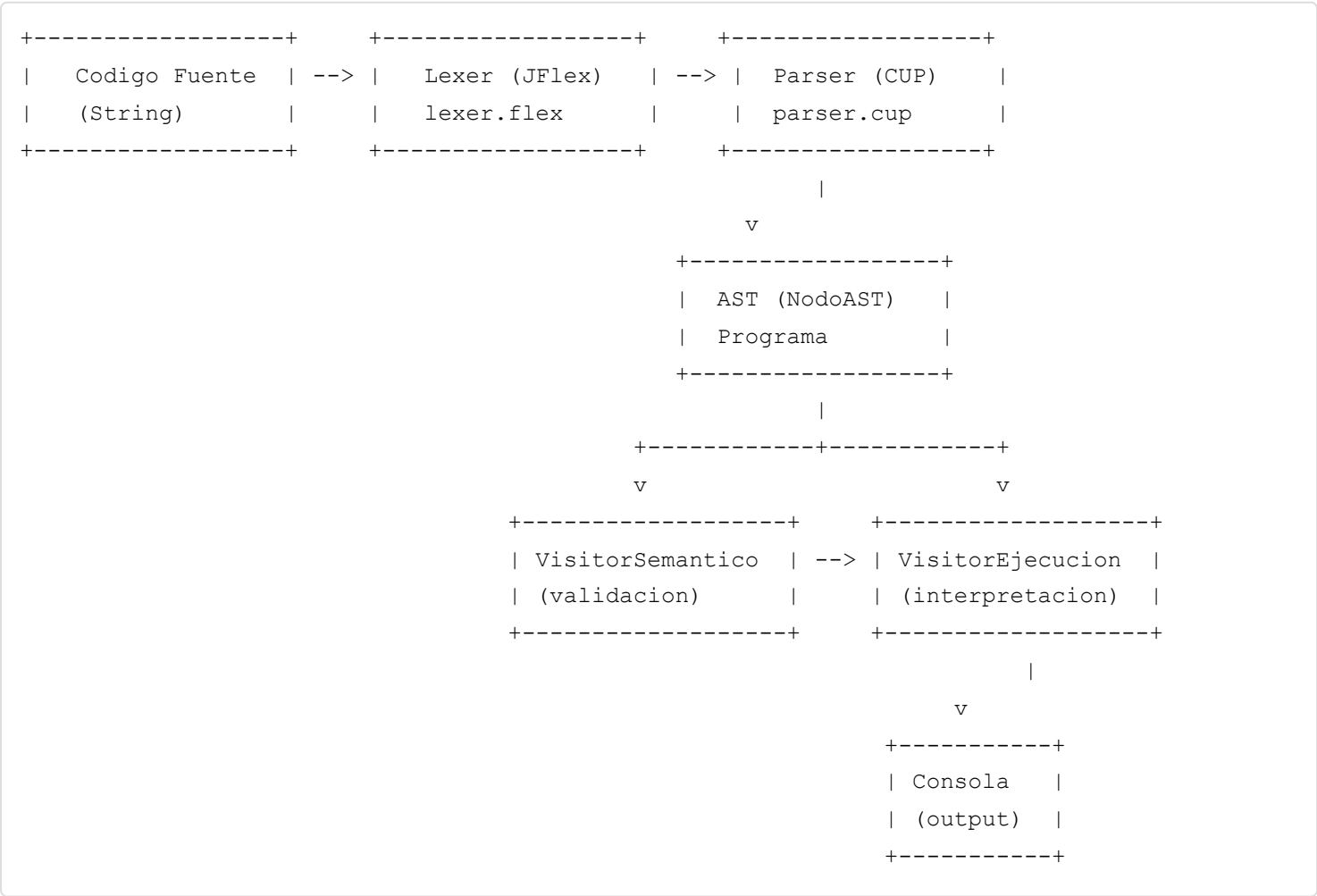
- 1. **Analisis Lexico:** Convierte el codigo fuente en una secuencia de tokens.
- 2. **Analisis Sintactico:** Construye un Arbol de Sintaxis Abstracta (AST) a partir de los tokens.
- 3. **Analisis Semantico:** Valida tipos, ambitos y reglas semanticas del lenguaje.
- 4. **Ejecucion:** Interpreta el AST y produce la salida del programa.

1.1 Caracteristicas del Lenguaje (Fase 1)

Caracteristica	Descripcion
Identificadores	Case sensitive, formato <code>[a-zA-Z_][a-zA-Z0-9_]*</code>

Característica	Descripcion
Comentarios	Linea // y bloque /* */ (anidables)
Tipos estaticos	int, float64, string, bool, rune
Nil	Valor nulo asignable a cualquier tipo
Variables	var id tipo = expr, var id tipo, id := expr
Asignacion	=, +=, -=
Operadores aritmeticos	+, -, *, /, %
Comparacion	==, !=, >, >=, <, <=
Logicos	&&, ^
If-Else	Incluye else if
For	for cond { } y for init; cond; update { }
Break/Continue	Solo dentro de ciclos for
Funciones	func main() como punto de entrada (opcional)
Embebidas	fmt.Println, strconv.Atoi, strconv.ParseFloat, reflect.TypeOf().String()

2. Arquitectura del Sistema



2.1 Flujo de Datos

- El usuario escribe codigo en `PanelEditor (RSyntaxTextArea)`.
- Al presionar "Ejecutar", `VentanaPrincipal.run()` crea el `Lexer` y el `parser`.
- El parser produce un AST (`Programa`).
- `VisitorSemantico` recorre el AST validando tipos y ambitos.

- 5. `VisitorEjecucion` recorre el AST ejecutando las instrucciones.
- 6. La salida se muestra en `PanelConsola`.
- 7. Los errores se recolectan en listas durante todo el proceso y se muestran via `ReporteDialog`.

3. Analisis Lexico (JFlex)

3.1 Archivo: `lexer.flex`

El analizador lexico es generado por JFlex a partir del archivo `lexer.flex`. Se define en el paquete `analisis` y genera la clase `Lexer`.

3.2 Estados del Lexer

Estado	Descripcion
YYINITIAL	Estado inicial, reconoce la mayoría de tokens
CADENA	Procesa literales de cadena "... " con escapes
COMENTARIO_LINEA	Ignora contenido hasta fin de linea //
COMENTARIO_BLOQUE	Ignora contenido hasta */ , soporta anidacion

3.3 Macros

```
digit      = [0-9]
letter     = [a-zA-Z]
underscore = "_"
id         = ({letter}|{underscore})({letter}|{digit}|{underscore})*
integer    = [0-9]+
decimal    = [0-9]+ "." [0-9]+
whitespace = [\ \r\t\f]+
newline    = [\n]
```

3.4 Palabras Reservadas

Token	Lexema
VAR	var
FUNC	func
MAIN	main
IF	if
ELSE	else
FOR	for
BREAK	break
CONTINUE	continue
INT	int
FLOAT64	float64
STRING	string
BOOL	bool
RUNE	rune
TRUE / FALSE	true / false
NIL	nil

3.5 Funciones Embebidas

Token	Lexema
FMT	fmt
STRCONV	strconv
REFLECT	reflect
PRINTLN	Println
TYPEOF	TypeOf
ATOI	Atoi
PARSEFLOAT	ParseFloat
STRINGFUNC	String

3.6 Operadores y Simbolos

Los operadores de multiples caracteres se reconocen antes que los de un solo caracter para evitar ambigüedades:

```
==  !=  >=  <=  +=  -=  :=  &&  ||  
>  <  +  -  *  /  %  =  !  (  )  {  }  ;  .  ,
```

3.7 Cadena de caracteres

Las cadenas se construyen en el estado `CADENA` acumulando caracteres en la variable `cadena` . Se soportan las siguientes secuencias de escape:

Secuencia	Significado
\\n	Nueva linea
\\r	Retorno de carro
\\t	Tabulacion
\\\\	Barra invertida
\\"	Comilla doble

Si una cadena no se cierra antes de un salto de linea, se registra un error lexico.

3.8 Recoleccion de Tokens

Cada regla del lexer invoca el metodo auxiliar `token(sym, lexema, tipo, linea, columna)` que:

1. Crea un `TokenInfo` con el lexema, tipo, linea y columna.
2. Lo agrega a la lista `tokens` del lexer.
3. Retorna un objeto `Symbol` para el parser.

```
private Symbol token(int sym, String lexema, String tipo, int line, int col) {  
    tokens.add(new TokenInfo(lexema, tipo, line, col));  
    return new Symbol(sym, line, col, lexema);  
}
```

3.9 Recoleccion de Errores Lexicos

Los caracteres no reconocidos en estado `YYINITIAL` y las cadenas no terminadas registran un `GoliteError` en la lista `errores` del lexer:

```
<YYINITIAL> [^]    { errores.add(new GoliteError("Lexico", "character no reconocido: " + yytext(  
<CADENA> \n        { errores.add(new GoliteError("Lexico", "cadena no terminada", yyline, yycol
```

4. Analisis Sintactico (CUP)

4.1 Archivo: `parser.cup`

El analizador sintactico es generado por CUP a partir del archivo `parser.cup` . Se define en el paquete `analisis` y genera la clase `parser` .

4.2 Precedencia y Asociatividad

La precedencia de operadores se define de menor a mayor:

```
left OR  
left AND  
left IGUALIGUAL, DIFERENTE  
left MAYOR, MAYORIGUAL, MENOR, MENORIGUAL  
left MAS, MENOS  
left MULT, DIV, MOD  
right NOT  
right UMENOS
```

4.3 No Terminales Principales

No Terminal	Descripcion
PROGRAMA	Raiz del AST, puede ser <code>func main()</code> o bloque directo
FUNCION_MAIN	Funcion principal <code>func main() { ... }</code>
BLOQUE	Bloque de instrucciones <code>{ ... }</code>
INSTRUCCIONES	Lista de instrucciones (recursiva)
INSTRUCCION	Una instruccion: declaracion, asignacion, if, for, break, continue, bloque, expresion
EXPRESION	Expresion con operadores, literales, identificadores, llamadas
TIPO	Tipo de dato: int, float64, string, bool, rune

4.4 Reglas de Produccion Principales

```
PROGRAMA ::= FUNCION_MAIN  
          | { INSTRUCCIONES }  
  
FUNCION_MAIN ::= func main ( ) BLOQUE  
  
BLOQUE ::= { INSTRUCCIONES }
```

```

INSTRUCCION ::= DECLARACION_VAR
              | ASIGNACION
              | ASIGNACION_OP
              | SENTENCIA_IF
              | SENTENCIA_FOR
              | SENTENCIA_BREAK
              | SENTENCIA_CONTINUE
              | BLOQUE
              | EXPRESION
              | error LLAVE2      (recuperacion de errores)

DECLARACION_VAR ::= var IDENTIFICADOR TIPO = EXPRESION
                 | var IDENTIFICADOR TIPO
                 | IDENTIFICADOR := EXPRESION

ASIGNACION ::= IDENTIFICADOR = EXPRESION

ASIGNACION_OP ::= IDENTIFICADOR += EXPRESION
                | IDENTIFICADOR -= EXPRESION

SENTENCIA_IF ::= if EXPRESION BLOQUE
               | if EXPRESION BLOQUE else BLOQUE
               | if EXPRESION BLOQUE else SENTENCIA_IF

SENTENCIA_FOR ::= for EXPRESION BLOQUE
                | for INSTRUCCION ; EXPRESION ; INSTRUCCION BLOQUE

SENTENCIA_BREAK ::= break
SENTENCIA_CONTINUE ::= continue

```

4.5 Recuperacion de Errores

El parser utiliza la produccion `error LLAVE2` para recuperarse de errores sintacticos. Cuando encuentra un token inesperado, descarta tokens hasta encontrar `}` y continua el analisis. Los errores se registran en la lista `errors` del parser:

```

public void syntax_error(Symbol s) {
    errors.add(new GoliteError("Sintactico", "simbolo inesperado: " + s.value, s.left, s.right));
}

```

5. Arbol de Sintaxis Abstracta (AST)

5.1 Interfaz Base: `NodoAST`

```

public interface NodoAST {
    <T> T accept(Visitor<T> visitor);
}

```

Todos los nodos del AST implementan esta interfaz, implementando el patron de diseno **Visitor**. Cada nodo tiene una clase interna `Context` que expone sus campos de forma inmutable para los visitors.

5.2 Nodos del AST (16 nodos)

Paquete	Clase	Campos
analisis.ast	Programa	FuncionMain main O List<NodoAST> instrucciones
analisis.ast	FuncionMain	Bloque bloque
analisis.ast	Bloque	List<NodoAST> instrucciones
analisis.ast.exp	ExpBinaria	NodoAST izquierdo , String operador , NodoAST derecho
analisis.ast.exp	ExpUnaria	String operador , NodoAST expr
analisis.ast.exp	Identificador	String nombre
analisis.ast.exp	Literal	Object valor , String tipo
analisis.ast.stm	DeclaracionVar	String id , String tipo , NodoAST expr
analisis.ast.stm	Asignacion	String id , NodoAST expr
analisis.ast.stm	AsignacionOp	String id , String operador , NodoAST expr
analisis.ast.stm	If	NodoAST condicion , Bloque bloqueIf , NodoAST bloqueElse
analisis.ast.stm	For	NodoAST init , NodoAST condicion , NodoAST update , Bloque bloque
analisis.ast.stm	Break	-
analisis.ast.stm	Continue	-
analisis.ast.stm	LlamadaFuncion	String paquete , String funcion , List<NodoAST> argumentos , boolean llamarString

5.3 Patron Context

Cada nodo expone sus datos a traves de una clase interna `Context` . Por ejemplo:

```
// Nodo: ExpBinaria
public static class Context {
    public final NodoAST izquierdo;
    public final String operador;
    public final NodoAST derecho;
    public final int linea;
    public final int columna;
}

// El nodo crea el Context y lo pasa al visitor:
@Override
public <T> T accept(Visitor<T> visitor) {
    return visitor.visit(new Context(this));
}
```

5.4 Interfaz Visitor

```
public interface Visitor<T> {
    T visit(Programa.Context ctx);
    T visit(FuncionMain.Context ctx);
    T visit(Bloque.Context ctx);
    T visit(DeclaracionVar.Context ctx);
}
```

```
T visit(Asignacion.Context ctx);
T visit(AsignacionOp.Context ctx);
T visit(If.Context ctx);
T visit(For.Context ctx);
T visit(Break.Context ctx);
T visit(Continue.Context ctx);
T visit(LlamadaFuncion.Context ctx);
T visit(ExpBinaria.Context ctx);
T visit(ExpUnaria.Context ctx);
T visit(Literal.Context ctx);
T visit(Identificador.Context ctx);
}
```

6. Analisis Semantico (VisitorSemantico)

6.1 Clase: VisitorSemantico

Implementa `Visitor<Void>` y recorre el AST para validar reglas semanticas. Sus componentes principales:

6.2 Tabla de Simbolos

TipoDato (enum)

```
public enum TipoDato {
    INT(0),
    FLOAT64(0.0),
    STRING(""),
    BOOL(false),
    RUNE(0),
    NIL(null);
}
```

Cada constante tiene un `valorDefecto` asociado.

Simbolo

```
public class Simbolo {
    public String id;
    public TipoDato tipo;
    public Object valor;
    public int linea;
    public int columna;
    public String ambito; // "global" o "local"
}
```

TablaSimbolos

Maneja ambitos anidados mediante una `List<Map<String, Simbolo>>`. Metodos principales:

Metodo	Descripcion
<code>nuevoAmbito()</code>	Agrega un nuevo ambito (hash map) al stack
<code>cerrarAmbito()</code>	Elimina el ambito actual del stack
<code>insertar(Simbolo)</code>	Inserta un simbolo en el ambito actual
<code>buscar(String id)</code>	Busca un simbolo desde el ambito actual hacia afuera
<code>existeEnAmbitoActual(String id)</code>	Verifica si existe en el ambito mas interno
<code>getNivelAmbito()</code>	Retorna el nivel actual (0 = global)

6.3 Validaciones Semanticas

Nodo	Validaciones
<code>DeclaracionVar</code>	Variable ya declarada en el ambito, compatibilidad de tipos entre declaracion e inicializacion
<code>Asignacion</code>	Variable declarada, compatibilidad de tipos
<code>AsignacionOp</code>	Variable declarada, tipos numericos (o string para <code>+=</code>)
<code>If</code>	Condicion debe ser <code>bool</code>
<code>For</code>	Condicion debe ser <code>bool</code> (si existe)
<code>Break / Continue</code>	Deben estar dentro de un ciclo <code>for</code>
<code>ExpBinaria</code>	Tipos compatibles para cada operador: <code>+</code> (numericos o string), <code>-</code> <code>*</code> <code>/</code> (numericos), <code>%</code> (int), <code>==</code> <code>!=</code> (mismo tipo), relacionales (numericos), logicos (bool)
<code>ExpUnaria</code>	<code>-</code> solo numericos, <code>!</code> solo bool
<code>Identificador</code>	Variable debe estar declarada
<code>LlamadaFuncion</code>	<code>strconv.Atoi/ParseFloat</code> esperan string como argumento

6.4 Promocion de Tipos

Cuando una operacion binaria mezcla `int` y `float64`, el resultado es `float64`:

```
private TipoDato promover(TipoDato a, TipoDato b) {
    if (a == TipoDato.FLOAT64 || b == TipoDato.FLOAT64) {
        return TipoDato.FLOAT64;
    }
    return TipoDato.INT;
}
```

6.5 Compatibilidad de Tipos

Se permite asignar `int` a una variable de tipo `float64` (promocion implicita):

```
private boolean sonCompatibles(TipoDato declarado, TipoDato expr) {
    if (declarado == expr) return true;
    if (declarado == TipoDato.FLOAT64 && expr == TipoDato.INT) return true;
    return false;
}
```

7. Ejecucion (VisitorEjecucion)

7.1 Clase: VisitorEjecucion

Implementa `Visitor<Valor>` y recorre el AST ejecutando las instrucciones. La salida del programa se acumula en el campo `public String output`.

7.2 Gestor de Ambitos

La clase `GestorAmbitos` maneja el stack de ambitos en tiempo de ejecucion usando un `Deque<Map<String, Valor>>`:

Metodo	Descripcion
<code>entrarBloque()</code>	Agrega un nuevo ambito vacio
<code>salirBloque()</code>	Elimina el ambito actual
<code>declarar(String id, Valor valor)</code>	Declara una variable en el ambito actual
<code>buscar(String id, int linea)</code>	Busca valor desde el ambito actual hacia afuera
<code>asignar(String id, Valor valor, int linea)</code>	Modifica el valor de una variable existente
<code>existeEnAlgunAmbito(String id)</code>	Verifica existencia en cualquier ambito

7.3 Flujo de Control

Las sentencias `break` y `continue` se implementan mediante excepciones unchecked:

```
class BreakException extends RuntimeException {}
class ContinueException extends RuntimeException {}
```

El ciclo `for` las captura para controlar el flujo:

```
public Valor visit(For.Context ctx) {
    if (ctx.init != null) ctx.init.accept(this);
    try {
        while (true) {
            Valor cond = ctx.condicion.accept(this);
            if (cond instanceof ValorBool b) {
                if (!b.valor()) break;
            }
            try {
                ctx.bloque.accept(this);
            } catch (ContinueException e) { }
            if (ctx.update != null) ctx.update.accept(this);
        }
    } catch (BreakException e) { }
    return defaultVoid;
}
```

7.4 Funciones Embebidas

fmt.Println

Concatena los argumentos separados por espacio y los imprime con salto de linea:

```
case "fmt":
    if (ctx.funcion.equals("Println")) {
        StringBuilder sb = new StringBuilder();
        for (int i = 0; i < ctx.argumentos.size(); i++) {
            if (i > 0) sb.append(" ");
            sb.append(ctx.argumentos.get(i).accept(this).toString());
        }
        output += sb.toString() + "\n";
    }
```

strconv.Atoi

Convierte un string a int:

```
try {
    int val = Integer.parseInt(vs.valor());
    return new ValorInt(val, ctx.linea, ctx.columna);
} catch (NumberFormatException e) {
    throw new RuntimeException("strconv.Atoi: '" + vs.valor() + "' no es un entero valido");
}
```

strconv.ParseFloat

Convierte un string a float64:

```
try {
    double val = Double.parseDouble(vs.valor());
    return new ValorFloat(val, ctx.linea, ctx.columna);
} catch (NumberFormatException e) {
    throw new RuntimeException("strconv.ParseFloat: '" + vs.valor() + "' no es un decimal val");
}
```

reflect.TypeOf().String()

Retorna el nombre del tipo de un valor como string:

```
case "reflect":
    Valor expr = ctx.argumentos.get(0).accept(this);
    if (ctx.llamarString) {
        return new ValorString(expr.obtenerTipoNombre(), ctx.linea, ctx.columna);
    }
```

7.5 Operaciones Aritmeticas y Logicas

La clase `Operaciones` contiene metodos estaticos para todas las operaciones:

Metodo	Operacion
<code>sumar(Valor a, Valor b, int linea)</code>	<code>+</code> (numericos y string)
<code>restar(Valor a, Valor b, int linea)</code>	<code>-</code>
<code>multiplicar(Valor a, Valor b, int linea)</code>	<code>*</code>
<code>dividir(Valor a, Valor b, int linea)</code>	<code>/</code> (valida division por cero)
<code>modulo(Valor a, Valor b, int linea)</code>	<code>%</code> (solo int, valida modulo por cero)
<code>comparar(Valor a, Valor b, int linea)</code>	<code>==</code> (todos los tipos)
<code>compararOrd(Valor a, Valor b, String op, int linea)</code>	<code>></code> , <code>>=</code> , <code><</code> , <code><=</code> (numericos)
<code>logicAnd(Valor a, Valor b, int linea)</code>	<code>&&</code>
<code>logicOr(Valor a, Valor b, int linea)</code>	<code>,</code>

Todas las operaciones usan *pattern matching* de Java 21 (`switch` con `when` y `case`) para determinar los tipos en tiempo de ejecucion.

8. Sistema de Valores en Tiempo de Ejecucion

8.1 Interfaz Sellada: `Valor`

```
public sealed interface Valor permits ValorInt, ValorFloat, ValorBool, ValorString, ValorRune
{
    int linea();
    int columna();
    String obtenerTipoNombre();
}
```

8.2 Implementaciones

Clase	Tipo	Almacena	<code>toString()</code>
<code>ValorInt</code>	<code>int</code>	<code>int valor</code>	<code>String.valueOf(valor)</code>
<code>ValorFloat</code>	<code>float64</code>	<code>double valor</code>	<code>String.format("%.1f", valor)</code> si es entero, sino <code>String.valueOf(valor)</code>
<code>ValorBool</code>	<code>bool</code>	<code>boolean valor</code>	<code>String.valueOf(valor)</code>
<code>ValorString</code>	<code>string</code>	<code>String valor</code>	<code>valor</code>
<code>ValorRune</code>	<code>rune</code>	<code>int valor</code>	<code>String.valueOf((char) valor)</code>
<code>ValorNil</code>	<code>nil</code>	<code>-</code>	<code>"nil"</code>
<code>ValorVoid</code>	<code>void</code>	<code>-</code>	<code>"void"</code>

Todas son **records** de Java 21 (`public record ValorInt(int valor, int linea, int columna) implements Valor`), lo que proporciona inmutabilidad, `equals/hashCode` y acceso a componentes de forma gratuita.

9. Manejo de Errores

9.1 Clase: `GoliteError`

```
public class GoliteError {
    private final String tipo;          // "Lexico", "Sintactico", "Semantico", "Ejecucion"
    private final String descripcion;
    private final int linea;
    private final int columna;
}
```

9.2 Recoleccion de Errores

Fase	Donde se recolecta	Tipo
Lexica	<code>Lexererrores</code> (List)	"Lexico"
Sintactica	<code>parser.errors</code> (List)	"Sintactico"
Semantica	<code>VisitorSemantico.errores</code> (List)	"Semantico"
Ejecucion	Excepciones capturadas en <code>VentanaPrincipal.run()</code>	"Ejecucion"

Los errores lexicos y sintacticos **no detienen** el proceso de compilacion. El sistema recolecta todos los errores encontrados y los muestra al usuario, permitiendo multiples errores por ejecucion.

9.3 Estrategia de Recuperacion de Errores Sintacticos

En el parser, la produccion:

```
INSTRUCCION ::= error LLAVE2
```

Permite que al encontrar un token inesperado, el parser descarte tokens hasta la siguiente `}` y continúe analizando, evitando que un solo error impida detectar errores posteriores.

10. Interfaz Grafica (GUI)

10.1 Paquete: `usac.compil.gui`

10.2 `VentanaPrincipal` (JFrame)

Componente principal que organiza la interfaz:

Menu Bar:

- **Archivo:** Nuevo (limpiar editor), Salir
- **Ejecutar:** Inicia el pipeline de compilacion
- **Reportes:** Reporte de tokens, Reporte de errores
- **Limpiar consola:** Borra la salida
- **Ayuda:** Acerca de

Layout:

```

+-----+
| Menu Bar                                |
+-----+
|                                         |
|  PanelEditor (RSyntaxTextArea)         |
|  (67% del espacio)                     |
|                                         |
+-----+
|  PanelConsola (JTextArea)               |
|  (33% del espacio)                     |
+-----+

```

10.3 PanelEditor

Utiliza **RSyntaxTextArea 3.3.4** con sintaxis Go, soporte para:

- Coloreado sintactico para Go
- Plegado de codigo (code folding)
- Auto-indentacion
- Barra de estado con numero de linea y columna

10.4 PanelConsola

Area de texto no editable con tema oscuro (fondo #282828, texto #C8C8C8), donde se muestra:

- La salida del programa ejecutado
- Mensajes de error
- Banner predeterminado

10.5 Ciclo de Compilacion (`run()`)

```

private void run() {
    String codigo = editor.obtenerTexto();
    lexer = new Lexer(new StringReader(codigo));
    parser = new parser(lexer);
    Symbol resultado = parser.parse();
    Programa programa = (Programa) resultado.value;

    VisitorSemantico semantico = new VisitorSemantico();
    programa.accept(semantico);

    VisitorEjecucion ejecucion = new VisitorEjecucion();
    programa.accept(ejecucion);

    consola.append(ejecucion.output);
}

```

11. Reportes

11.1 ReporteDialog

Dialogo reutilizable que muestra datos tabulares en un `JTable` :

```
public class ReporteDialog extends JDialog {
    public ReporteDialog(JFrame parent, String titulo, String[] columnas, List<String[]> fila
        // Construye JTable con DefaultTableModel no editable
        // Muestra en JScrollPane
    }
}
```

11.2 Reporte de Tokens

Muestra todos los tokens reconocidos por el lexer:

No.	Lexema	Tipo	Linea	Columna
1	var	var	1	1
2	x	id	1	5
3	10	int	1	11

Los tokens se recolectan durante el analisis lexico mediante el metodo `token()` .

11.3 Reporte de Errores

Muestra todos los errores recolectados durante la compilacion:

No.	Descripcion	Linea	Columna	Tipo
1	Variable 'x' no declarada	3	5	Semantico
2	Caracter no reconocido: @	1	1	Lexico

Los errores se consolidan de las listas del lexer y el parser.

12. Gramatica en BNF

```
<PROGRAMA> ::= <FUNCION_MAIN>
              | "{" <INSTRUCCIONES> "}"

<FUNCION_MAIN> ::= "func" "main" "(" ")" <BLOQUE>

<BLOQUE> ::= "{" <INSTRUCCIONES> "}"

<INSTRUCCIONES> ::= <INSTRUCCIONES> <INSTRUCCION>
                  | <INSTRUCCION>

<INSTRUCCION> ::= <DECLARACION_VAR>
```

```

| <ASIGNACION>
| <ASIGNACION_OP>
| <SENTENCIA_IF>
| <SENTENCIA_FOR>
| "break"
| "continue"
| <BLOQUE>
| <EXPRESION>
| "error" "}"

<DECLARACION_VAR> ::= "var" <IDENTIFICADOR> <TIPO> "=" <EXPRESION>
| "var" <IDENTIFICADOR> <TIPO>
| <IDENTIFICADOR> ":" "=" <EXPRESION>

<ASIGNACION> ::= <IDENTIFICADOR> "=" <EXPRESION>

<ASIGNACION_OP> ::= <IDENTIFICADOR> "+=" <EXPRESION>
| <IDENTIFICADOR> "-=" <EXPRESION>

<SENTENCIA_IF> ::= "if" <EXPRESION> <BLOQUE>
| "if" <EXPRESION> <BLOQUE> "else" <BLOQUE>
| "if" <EXPRESION> <BLOQUE> "else" <SENTENCIA_IF>

<SENTENCIA_FOR> ::= "for" <EXPRESION> <BLOQUE>
| "for" <INSTRUCCION> ";" <EXPRESION> ";" <INSTRUCCION> <BLOQUE>

<LLAMADA_FUNCION> ::= "fmt" "." "Println" "(" <LISTA_EXPRESIONES> ")"
| "strconv" "." "Atoi" "(" <EXPRESION> ")"
| "strconv" "." "ParseFloat" "(" <EXPRESION> ")"
| "reflect" "." "TypeOf" "(" <EXPRESION> ")" "." "String" "(" " ")"

<LISTA_EXPRESIONES> ::= <LISTA_EXPRESIONES> "," <EXPRESION>
| <EXPRESION>

<TIPO> ::= "int" | "float64" | "string" | "bool" | "rune"

<EXPRESION> ::= <EXPRESION> "||" <EXPRESION>
| <EXPRESION> "&&" <EXPRESION>
| <EXPRESION> "==" <EXPRESION>
| <EXPRESION> "!=" <EXPRESION>
| <EXPRESION> ">" <EXPRESION>
| <EXPRESION> ">=" <EXPRESION>
| <EXPRESION> "<" <EXPRESION>
| <EXPRESION> "<=" <EXPRESION>
| <EXPRESION> "+" <EXPRESION>
| <EXPRESION> "-" <EXPRESION>
| <EXPRESION> "*" <EXPRESION>
| <EXPRESION> "/" <EXPRESION>
| <EXPRESION> "%" <EXPRESION>
| "-" <EXPRESION> (unario, %prec UMENOS)
| "!" <EXPRESION>
| <ENTERO>
| <DECIMAL>

```



```
| <CADENA>
| <CHAR>
| "true" | "false"
| "nil"
| <IDENTIFICADOR>
| "(" <EXPRESION> ")"
| <LLAMADA_FUNCION>
```

```
<IDENTIFICADOR> ::= [a-zA-Z_][a-zA-Z0-9_]*
<ENTERO> ::= [0-9]+
<DECIMAL> ::= [0-9]+\.[0-9]+
<CADENA> ::= "'" <CARACTERES> "'"
<CHAR> ::= "'" <CHARACTER> "'" | "'\\\" <ESCAPE> "'"
```

13. Compilacion y Ejecucion

13.1 Requisitos

- Java 21 o superior
- Apache Maven 3.x
- JFlex 1.9.1 (manejado por Maven)
- CUP 11b vbmacher (manejado por Maven)

13.2 Compilacion

```
# Compilar sin ejecutar pruebas
mvn clean compile -q

# Compilar y empaquetar JAR
mvn clean package

# Compilar y ejecutar
mvn clean compile exec:java
```

13.3 Ejecucion

```
# Ejecutar el JAR
java -jar Compilador/target/Compilador-1.0-SNAPSHOT.jar

# O directamente con Maven
mvn exec:java
```

13.4 Dependencias (Maven)

```
<dependency>
  <groupId>com.fifesoft</groupId>
  <artifactId>rsyntaxtextarea</artifactId>
  <version>3.3.4</version>
</dependency>
```

13.5 Plugins de Build

```
<plugin>
  <groupId>com.github.vbmacher</groupId>
  <artifactId>cup-maven-plugin</artifactId>
  <version>11b-2017.05.02</version>
</plugin>
<plugin>
  <groupId>de.jflex</groupId>
  <artifactId>jflex-maven-plugin</artifactId>
  <version>1.9.1</version>
</plugin>
```

El plugin de JFlex se ejecuta antes que el de CUP en la fase `generate-sources` , y CUP antes de la compilacion.

14. Estructura del Proyecto

```
Compilador/
├─ pom.xml
├─ src/
│   └─ main/
│       ├── cup/
│       │   └─ parser.cup                # Gramatica y acciones
│       ├── jflex/
│       │   └─ lexer.flex                # Definicion lexica
│       └─ java/
│           └─ analisis/
│               ├── Compilador.java      # Punto de entrada
│               ├── ast/
│               │   ├── NodoAST.java     # Interfaz base del AST
│               │   ├── Programa.java
│               │   ├── FuncionMain.java
│               │   ├── Bloque.java
│               │   └─ exp/
│               │       ├── ExpBinaria.java
│               │       ├── ExpUnaria.java
│               │       ├── Identificador.java
│               │       └─ Literal.java
│               └─ stm/
│                   ├── Asignacion.java
│                   └─ AsignacionOp.java
```

```
| | | Break.java
| | | Continue.java
| | | DeclaracionVar.java
| | | For.java
| | | If.java
| | | LlamadaFuncion.java
| | semantic/
| | | TipoDato.java
| | | Simbolo.java
| | | TablaSimbolos.java
| | visitor/
| | | Visitor.java
| | | VisitorSemantico.java
| | | ejecucion/
| | | | BreakException.java
| | | | ContinueException.java
| | | | GestorAmbitos.java
| | | | Operaciones.java
| | | | VisitorEjecucion.java
| | | | valor/
| | | | | Valor.java
| | | | | ValorBool.java
| | | | | ValorFloat.java
| | | | | ValorInt.java
| | | | | ValorNil.java
| | | | | ValorRune.java
| | | | | ValorString.java
| | | | | ValorVoid.java
| | usac/
| | | compil/
| | | | gui/
| | | | | PanelConsola.java
| | | | | PanelEditor.java
| | | | | VentanaPrincipal.java
| | | | | reports/
| | | | | | GoliteError.java
| | | | | | ReporteDialog.java
| | | | | | TokenInfo.java
```

PLAN_FASE_1.md

MANUAL_TECNICO.md